

# Given the Kernel, What Is the Research Program? An Affordance Survey of the Typed Claim Graph for Synthesis

## 1. Executive Summary

The physics-kernel research program operates upon a strictly defined typed claim graph characterized by immutable mechanical invariants. These invariants enforce append-only provenance, deterministic grade bounding, the first-class tracking of withdrawn claims and open gaps, and the suspension of epistemic contradictions without forced resolution. By inverting the standard system-design methodology—asking not what tooling the research program requires, but rather what novel research program the built tool makes possible—this report surveys precedent fields spanning formal verification, database theory, scientometrics, abstract argumentation, and safety engineering. The objective is to identify historical structural isomorphisms matching the kernel's invariants and to derive verifiable, load-bearing affordances for research synthesis.

The analysis reveals three highly promising affordances that require no new infrastructure, representing "pure reads" over the existing graph. The strongest is *Dynamic Shepardizing*, drawn from the legal citation networks of the nineteenth century. This affordance utilizes the graph's provenance structure to mechanically flag and propagate the epistemic taint of superseded or withdrawn claims down the entire dependency cone, preventing the synthesis of theories built upon invalidated axioms. The second is the *Materiality-Weighted Audit Queue*, adapted from financial audit sampling theory. This mechanism dramatically reduces the cognitive load placed upon the human director by algorithmically ranking unverified claims according to their downstream structural risk, formalizing the allocation of expert attention. The third is *Incremental Synthesis View Maintenance*, borrowed from relational database theory. This approach allows literature reviews and theoretical reconciliations to function as materialized views that automatically degrade in earned grade or trigger localized LLM recomputation when an underlying assumption is invalidated, eliminating the need for manual, corpus-wide re-evaluation.

Conversely, the survey strongly indicates that the program's prior inclination toward *Staked Verification*—specifically the integration of prediction markets and replication-style betting—is severely overrated in the context of theoretical physics. While betting and replication markets show strong predictive accuracy in the social sciences, historical evidence dictates that these mechanisms rely fundamentally on rapid, unambiguous settlement criteria. The inherently slow and theoretically ambiguous settlement latency of frontier physics renders traditional staking mechanisms economically inviable, classifying this affordance as effectively dead without profound structural refinement.

## 2. Precedent Harvest

The following analysis isolates each mechanical invariant of the physics-kernel, identifies distinct disciplines that have operated under identical structural constraints, and reports the

synthesis tooling that organically emerged from those constraints. By extracting the load-bearing preconditions of each historical tool, the analysis maps these precedents back to the kernel as candidate affordances, explicitly noting the disanalogies that threaten their transferability.

## I1. Typed Claims and Deterministic Grading

The first invariant dictates that every claim possesses a specific kind and a corresponding grade ceiling, with an earned grade computed deterministically from its support structure. Declared grades exceeding the earned grade are mechanically rejected by the system.

In the field of epidemiological evidence synthesis, the GRADE (Grading of Recommendations Assessment, Development and Evaluation) framework operates under a nearly identical structural property. GRADE explicitly types evidence, assigning high initial ceilings to randomized controlled trials and low initial ceilings to observational studies. The system then applies deterministic, rule-based algorithms for downgrading or upgrading confidence levels. A body of evidence is mechanically penalized by one or two levels for specific, documented failures such as risk of bias, inconsistency across trials, indirectness of the population, or statistical imprecision. The synthesis tooling that emerged from this invariant is the "Summary of Findings" table, generated by platforms like GRADEpro, which enforces the strict rule that a clinical recommendation's strength is absolutely bound by the lowest-graded critical outcome supporting it. The load-bearing precondition in epidemiology is a broad, institutional consensus on what constitutes a methodological flaw. The affordance for the physics-kernel is a *Deterministic Downgrade Engine*. Within this engine, a high-level theoretical synthesis generated by an LLM is automatically downgraded by the graph if its weakest load-bearing theoretical assumption falls below a required confidence threshold. The primary disanalogy is that physics lacks the standardized statistical heterogeneity tests—such as Cochran's Q or the  $I^2$  statistic—used in GRADE to mechanically identify trial inconsistency. Consequently, the kernel must rely on logical, rather than statistical, bounding algorithms.

In the domain of formal verification and proof assistants, the Lean mathematical library (Mathlib) treats theorems as typed claims where the grading system is effectively binary. A proof is either formally verified by the Lean kernel, or it relies on a sorry placeholder, indicating an unproven gap. The tooling that emerged to support this structure includes rigorous linters that mechanically reject code failing to meet structural standards, ensuring that a theorem cannot declare itself proven if its dependency tree contains unverified axioms. The precondition is the existence of a machine-checkable formal language capable of expressing absolute mathematical certainty. Mapped to the physics-kernel, the affordance is *Strict Axiomatic Bounding*. This affordance prevents LLM-generated synthesis products from asserting a high earned grade if the graph trace reveals a hidden dependency on informal or unverified premises. The disanalogy is that Lean deals in absolute mathematical proof, whereas the physics-kernel must accommodate probabilistic, empirical, or heuristic support, requiring the kernel's grading system to operate along a continuous spectrum rather than as a binary state switch.

## I2. The Append-Only Store

The second invariant establishes an append-only architecture where statements are superseded but never edited, and citing a superseded statement is mechanically rejected. In software engineering, version control systems—most notably Git—rely on an append-only

directed acyclic graph of commits. The analytical tooling that grew from this structural invariant includes utilities like git bisect and git blame. The git bisect tool utilizes binary search through the append-only history to mechanically isolate the exact commit that introduced a defect, compilation failure, or logical contradiction. Similarly, git blame allows developers to trace the exact provenance, authorial identity, and temporal context of any specific line of code. The load-bearing precondition is a linear, immutable history of state changes coupled with a deterministic test suite. The corresponding affordance in the kernel is *Automated Epistemic Bisection*. This mechanism allows the human director to mechanically isolate the exact claim insertion that caused a previously stable physical theory or synthesis to collapse into a contradictory state. The disanalogy is that software code execution is binary—it either passes or fails continuous integration tests—whereas theoretical physics synthesis may degrade in quality gradually without a hard compilation failure, making the precise bisection target more difficult to define.

In the realm of financial accounting and audit theory, the double-entry ledger acts as a foundational append-only store. Errors in accounting are never erased; they are corrected via new, reversing journal entries, preserving the complete audit trail in perpetuity. The analytical tooling that emerged here is the forensic audit, which leverages the immutability of the ledger to perform anomaly detection, trace the flow of capital over time, and reconstruct historical financial states. The precondition is a closed, balanced system of value transfer where every credit has a mathematically equal debit. Mapped to the kernel, the affordance is *Forensic Epistemic Auditing*. Because superseded and failed claims remain permanently visible in the graph, synthesis tools can query historical states to understand precisely why a prior research program failed. This prevents stateless LLM sessions from inadvertently proposing previously superseded theoretical pathways. The disanalogy is that epistemic value within a claim graph does not sum to zero like financial capital, rendering anomaly detection based on balancing equations impossible.

### 13. First-Class Kills and Reinstatement Conditions

The third invariant elevates "kills" to first-class entities: withdrawn claims carry characterized failure mechanisms and typed reinstatement conditions, and re-entry without meeting these conditions is rejected.

Safety engineering, specifically aviation incident tracking via the Aviation Safety Reporting System (ASRS), relies heavily on near-miss registries. These systems categorize failure mechanisms, contributing factors, and human errors without deleting the failed flight path from the operational record. The analytical tooling that emerged involves automated trend analysis algorithms that scan these categorized failures to identify systemic, environmental, or operational risks, leading to the generation of predictive safety alerts. The precondition is a standardized, highly regulated taxonomy of hazards. The affordance for the kernel is *Systemic Failure Trend Analysis*. By analyzing the localized failure margins and mechanisms of killed claims within the graph, synthesis LLMs can identify underlying systemic weaknesses in a specific theoretical approach. This allows the system to steer the research program away from entire classes of flawed hypotheses before they are fully generated. The disanalogy is that ASRS relies on human narrative categorization and manual coding, whereas the kernel requires machine-readable, deterministic failure margins to automate the analysis.

In evolutionary computation, Quality-Diversity (QD) algorithms such as MAP-Elites maintain an explicit, multi-dimensional archive of varied, often lower-fitness solutions. Rather than discarding failed or suboptimal mutations, MAP-Elites stores them in a grid based on distinct behavioral

descriptors, recognizing that low-fitness solutions often serve as critical "stepping stones" necessary to cross deceptive fitness landscapes. The precondition is a clearly defined behavioral space that allows for structural niching and continuous metric evaluation. Mapped to the physics-kernel, the affordance is *Stepping-Stone Recovery*. An LLM synthesizing a new theoretical framework can query the graph of killed claims, extract the exact failure mechanism, and utilize the "dead" claim as a foundational stepping stone to bridge a theoretical gap, provided the new contextual synthesis satisfies the original claim's typed reinstatement condition. The disanalogy is that evolutionary fitness in algorithms like MAP-Elites is computationally cheap to calculate, while reinstating a killed physics claim requires expensive, resource-intensive re-evaluation by the verification economy.

## 14. First-Class Open Gaps

The fourth invariant dictates that unclosed claims carry explicit closing conditions, and the set of all open gaps is instantly computable at any given moment.

In the history of mathematics, curated lists of open problems—most notably David Hilbert's 23 problems presented in 1900—have served as generative epistemic infrastructure. These open gaps did not merely represent missing knowledge; they provided a structured, highly visible roadmap that directed the allocation of global mathematical attention and funding for an entire century. The "tooling" in this context was the community's collective prioritization mechanism, where the explicit characterization of a gap's boundaries enabled localized, individual efforts to yield field-wide, generative progress. The precondition is a shared, absolute consensus on the significance of the problem and the rigorous criteria for its solution. The affordance for the kernel is *Algorithmic Attention Directing*. By treating open gaps as first-class, computable objects with explicit closing conditions, the kernel can deploy its LLM sessions exclusively toward generating proofs, derivations, or code for the highest-leverage gaps. This transforms passive unknowns into an active, optimized task queue. The disanalogy is that Hilbert's problems were curated by singular human genius and intuition; the kernel, conversely, must rely on mechanical dependency graphs to weight the systemic importance of a gap.

In open-source software development, issue trackers act as computable sets of open gaps. The tooling that emerged here includes triage automation, tag-based routing, and priority ranking systems that match developer expertise to specific bugs based on the issue's metadata. The precondition is a highly modular codebase where issues can be functionally isolated. The candidate affordance is *Targeted Session Deployment*. In this workflow, the human director configures stateless LLM sessions with highly specific system prompts tailored to the exact closing condition of a single gap. By isolating the context window to a single, computable gap rather than the entire corpus, the system minimizes hallucination and context exhaustion.

## 15. Provenance as a Graph

The fifth invariant states that all provenance is graph-based: sources carry lineage (*rests\_on*), audits carry checker identity, and footprint closure computably reveals when two audits share ancestry.

In legal informatics, the introduction of Shepard's Citations in 1873 established a massive, graph-based provenance system for case law. It explicitly tracked whether a legal precedent had been overruled, questioned, distinguished, or affirmed by subsequent judicial decisions. The analytical tool that emerged is "Shepardizing," a mandatory synthesis practice where a legal professional mechanically traces the lineage of a case to ensure its precedential authority

remains valid before citing it in a brief. The precondition is a hierarchical system of binding authority (*stare decisis*). The affordance for the kernel is *Dynamic Shepardizing of Physics Claims*. Before an LLM session is permitted to synthesize a set of claims into a new theoretical proposal, a cheap graph query traverses the `rests_on` lineage. This mechanical check ensures that no foundational premise has been subsequently killed or superseded by an audit footprint sharing ancestry with a known failure. The disanalogy is that legal precedent is overturned by human judicial decree and interpretation, whereas physics claims degrade through empirical falsification, mathematical contradiction, or deterministic recomputation.

In database theory, Provenance Semirings provide a rigorous mathematical framework for tracking the lineage of data through complex queries. By annotating base tuples with polynomials (e.g.,  $N[X]$ ), databases can track exactly which source records contributed to a specific query output, effectively computing the "why-provenance" of data across relational algebra operations like projection, selection, and natural join. The tooling includes mechanisms for incremental deletion propagation; if a source tuple is removed or modified, the semiring structure mathematically determines exactly which downstream materialized views must be updated or dropped. The precondition is the strict mathematical structure of commutative semirings applied to relational algebra. The mapped affordance is *Mechanical Taint Propagation*. If a specific automated checker or empirical data source is found to be unreliable, the semiring-like footprint closure of the kernel allows the system to instantly compute every downstream claim contaminated by that footprint. These claims are automatically flagged for re-audit or downgraded without requiring human review. The disanalogy is the immense semantic and mathematical complexity of theoretical physics claims compared to discrete relational database tuples.

## 16. Held Contradictions

The sixth invariant establishes that the system holds contradictions indefinitely without resolving them, computing crux candidates but never rendering final verdicts.

In abstract argumentation theory, Dung's Argumentation Frameworks model knowledge entirely through attack and defense relations between arguments, completely abstracting the internal logic of the arguments themselves. Contradictions are held indefinitely within the directed graph. The analytical tooling that emerged encompasses semantics for computing "extensions"—sets of arguments that can be collectively accepted because they defend themselves against outside attacks (e.g., grounded, preferred, or stable extensions). The system computes what stands based on topological defense, utilizing a characteristic function  $f_{\mathcal{A}}(S)$  to find fixed points of acceptability. The precondition is a clear binary definition of an "attack" relation. The affordance for the kernel is *Extension-Based Hypothesis Generation*. The system can use Dung semantics to compute multiple mutually exclusive, internally consistent "preferred extensions" of the physics graph. The human director can then spawn separate LLM sessions to explore each divergent theoretical branch simultaneously, acknowledging that the contradiction cannot yet be resolved. The disanalogy is that Dung frameworks assume attacks are absolute and binary, whereas the kernel relies on graded support and probabilistic falsification.

In formal logic and database management, Belnap's Four-Valued Logic was designed specifically for systems merging information from multiple, potentially conflicting sources. It expands classical logic to include four epistemic states: True, False, Neither (incomplete), and Both (contradictory). This paraconsistent approach prevents a single contradiction from causing a logical explosion (the principle of explosion, or *ex contradictione quodlibet*), where all statements within the system become provable. The tooling is a query engine that safely returns

results and continues operations even when the underlying data is conflicted. The precondition is a defined bilattice structure for truth values. The affordance is *Paraconsistent Synthesis*. The LLM sessions can be prompted to synthesize literature reviews over the entire corpus without failing upon encountering a contradiction; the graph simply passes the "Both" state upward, highlighting the crux for the human director to design an experiment around. The disanalogy is that physics ultimately demands a single unified truth about the natural world, whereas paraconsistent databases are designed to tolerate indefinite operational contradiction.

## 17. Computations in a Transformation Store

The seventh invariant isolates computations in a transformation store where transforms require assumptions, consume and emit values, but never independently emit standing.

In database architecture, Incremental View Maintenance (IVM) systems treat complex queries as materialized views that must be updated when source tables change. Rather than recomputing the entire view from scratch, the system applies transformations only to the delta of the input data. The tooling involves logical plan normalization, fingerprinting, and decomposition to calculate precise, minimal updates. The precondition is a closed algebraic system of relations. Mapped to the kernel, the affordance is *Assumption-Bound Recomputation*. Because transforms in the kernel require explicit assumption manifests, when a core physics parameter is updated, the transformation store can incrementally push the new value through all dependent mathematical models. This occurs without needing to re-evaluate the epistemic standing of the theoretical claims themselves, cleanly separating the execution of computation from the judgment of truth. The disanalogy is that database updates are deterministic data insertions, while physics transformations may involve complex, nonlinear mathematical models that resist simple algebraic delta updates.

## 18. Recomputable Standing and Forkable Epistemic State

The eighth invariant dictates that standing is publicly recomputable, the entire epistemic state serializes and forks, and a fork constitutes a legitimate exit from the system.

In open-source software ecosystems, code repositories are infinitely forkable. The analytical tooling that emerged is fork-diff analysis, which allows communities to analyze the divergence in logic, feature prioritization, and security patching between a main repository and its forks. The precondition is a standardized version control syntax and a decentralized developer base.

Mapped to the kernel, the affordance is *Adversarial Epistemic Forking*. Two rival LLM sessions or research teams can fork the exact same epistemic state, apply differing assumption manifests to resolve a held contradiction, and build out competing theoretical frameworks in isolation. The human director can then run a mechanical diff on the two forks to identify exactly which downstream predictions diverge, providing a perfect crux for experimental testing. The disanalogy is that software forks often aim to eventually merge back together via pull requests, whereas physics forks represent mutually exclusive models of reality that must eventually result in the falsification and kill of one branch.

## 19. Cheap Structural Queries

The ninth invariant establishes that dependency cones, contamination paths, and grade distributions are computationally cheap to query at any time.

In software engineering, static analysis tools and linters rely on the abstract syntax tree of a

codebase to perform cheap, instantaneous queries identifying type mismatches, unused variables, and security anti-patterns. The tooling reduces developer cognitive load by filtering mechanical errors out of the code review process, drastically reducing the time spent identifying trivial bugs. Studies have shown that advanced static analysis and typed mocking can reduce Diff Authoring Time (DAT) by up to 33%. The precondition is a rigorously defined syntax and deterministic rulesets. The affordance for the kernel is *Graph-Level Linting for LLM Outputs*. Before an LLM-generated claim is presented to the human director for review, a suite of structural queries automatically checks for circular dependencies, unsupported grade declarations, and reliance on superseded claims. Claims failing the linter are mechanically bounced back to the LLM session for revision, drastically reducing the human director's review burden. The disanalogy is that software linters check for syntactical correctness, whereas the kernel's linters must evaluate structural epistemic validity, raising the risk of "warning fatigue" if the rulesets are overly rigid.

### 3. Conjunction Analysis: Novelty and the Unprecedented

The kernel's distinctive power arises not from its isolated invariants, but from their unique conjunction. Analyzing systems that combine graded claims (I1), first-class kills (I3), computable audit independence (I5), and held contradictions (I6) is necessary to determine where the kernel represents genuinely novel infrastructure.

No exact historical precedent exists for this specific four-part conjunction. This boundary explicitly marks where the physics-kernel transitions from applied engineering into unprecedented epistemic territory. However, systems possessing three of these invariants provide critical insight into the synthesis practices that emerge near this boundary.

Wikipedia's dispute resolution machinery and editorial guidelines deal extensively with graded claims (e.g., featured articles versus stubs), first-class kills (reversions and article deletions), and held contradictions (Neutral Point of View policies requiring the presentation of multiple conflicting viewpoints). The synthesis tooling that emerged includes heavily codified arbitration processes and the tracking of editor edit-counts as a proxy for audit footprints. However, statistical analysis of Wikipedia's arbitration reveals a catastrophic failure mode: the dispute resolution process actively ignores the logical content of the disputes and focuses entirely on user conduct, edit warring, and social capital to quash conflicts. Because Wikipedia lacks I5—strict, logical provenance graphs—its synthesis practices degenerate into social consensus and "winner-take-all" ideological dominance.

The Lean Mathlib ecosystem possesses I1 (typed claims), I3 (first-class kills via a formal deprecation system), and I5 (strict dependency graphs). Lean's deprecation tooling allows lemmas to be marked obsolete, warning users and providing a grace period before breaking the build. The synthesis practice enabled here is massive, coordinated library refactoring, where thousands of interconnected proofs can be upgraded systematically. However, Lean strictly lacks I6; it cannot hold a formal contradiction. If a contradiction is introduced, the system halts, and the formal logic explodes.

**The Unprecedented Affordance:** Because the kernel successfully combines Lean's rigorous dependency tracking with Wikipedia's tolerance for held contradictions, an entirely novel synthesis practice becomes possible: *Topological Crux Isolation*. In standard systems, a contradiction either halts progress completely or requires a social consensus to override. In the kernel, a contradiction between two high-graded claims triggers a cheap query (I9) traversing

the independent audit trails (I5) of both claims down to their axiomatic roots. The system can mechanically compute the exact set of differing foundational assumptions that gave rise to the contradiction. The synthesis product is not a resolution, but a computationally guaranteed map of the ideological divergence—an affordance that cannot be imported from any precedent field because no historical field has sustained this exact structural conjunction.

## 4. Synthesis as a Typed Act

The research program's current paradigm treats synthesis as an intelligent LLM reading prose and making intuited connections. Under the kernel's invariants, a synthesis product—whether a literature review, a reconciliation, or a survey claim—must itself be treated as a typed, graded claim. It rests explicitly upon its citations, is contaminated by their weaknesses, and is superseded when they move (I1, I2, I9).

In the field of evidence-based medicine, the Cochrane Collaboration pioneered the "Living Systematic Review" (LSR). Traditional systematic reviews degrade in value as soon as new trials are published. LSRs are continuously updated, incorporating new evidence via explicit triggers—such as the publication of a new trial that changes the certainty of the overall effect, strengthens a conclusion, or introduces a new comparator. The load-bearing precondition for LSRs is a stable, continuous hosting infrastructure and pre-specified statistical rules for when a full update is justified.

In database architecture, Incremental View Maintenance (IVM) treats a synthesis (a query output) as a materialized view. When the underlying data changes, the view is not entirely re-queried; instead, the system applies algebraic transformations to update the view based only on the delta of the changing inputs, saving massive computational overhead.

Combining these precedents yields a powerful affordance for the kernel: *Mechanically Triggered Living Synthesis*. When a synthesis claim is authored by an LLM, it is instantiated in the graph with explicit dependencies on its cited claims. If any underlying claim is killed (I3), degraded (I1), or superseded (I2), the graph triggers an incremental re-evaluation. Rather than having the LLM re-read the entire corpus, the system passes only the specific delta (e.g., "Claim X has been downgraded from Level 4 to Level 2") to the LLM, prompting it to adjust the overarching synthesis claim accordingly.

Fields with weaker versions of this concept learned severe lessons regarding its failure modes. In systematic reviews, update triggers can cause "warning fatigue" if the threshold for notification is set too low, leading to endless minor revisions that consume vast resources but provide no actionable decision-making value. To survive in the physics-kernel, the affordance requires strict materiality thresholds: a synthesis LLM is only invoked for recomputation if the downstream grade of the synthesis claim is mathematically threatened by the upstream change.

## 5. Verifier Economy Analysis

The physics-kernel program faces a binding operational constraint: the cost per verified proposal, with the human director serving as the most expensive and scarce resource in the loop. The "Verifier Economy" shifts the analytical focus from generating claims to efficiently filtering, sorting, and verifying them.

In financial auditing, the concept of "materiality" formalizes the allocation of expert attention. Auditors do not verify every transaction; they rely on professional judgment and single-rule quantitative methods (e.g., 5% of pre-tax income, or 0.5% of total assets) to establish



thresholds. Only misstatements that could "reasonably be expected to influence the economic decisions of users" command expert audit attention. The precondition is a quantifiable metric of overall system value, such as revenue or total assets.

In software engineering, triage systems and static analysis linters operate as a pre-filter economy. By automatically flagging syntactical errors, style violations, and basic security flaws before code review, linters dramatically reduce developer review load. Studies at Meta utilizing the "Diff Authoring Time" (DAT) metric demonstrated that typed mocks and automated analysis yielded a 14% to 33% improvement in active developer work time. The precondition is the ability to encode team standards as deterministic rules.

Mapped to the kernel, these precedents offer the *Materiality-Weighted Audit Queue*. Currently, the human director must verify LLM outputs intuitively. The affordance applies cheap structural queries (I9) to calculate the "epistemic materiality" of any unverified claim. A claim's materiality is defined by the volume and grade of the downstream nodes that depend upon it. If a claim sits at the root of a massive dependency cone, it is highly material and is flagged for immediate, rigorous human review. If a claim is an isolated leaf node, it falls below the materiality threshold and is either auto-approved or relegated to low-priority, stateless LLM review. This mechanism demonstrably recovers director time by ensuring expensive human cognition is deployed exclusively where structural risk to the graph is highest.

## 6. Staked Verification Survey

The program is considering a mechanism that allows automated checkers to stake on nodes, resolving stakes via settlement events to build per-checker calibration track records. The viability of this mechanism rests entirely on the dynamics of settlement latency and unambiguity. In the social sciences, the SCORE project and related replication markets (led by researchers such as Camerer and Dreber) successfully used betting markets to predict the outcome of replication studies. Market participants bought and sold shares based on their belief that a published study would successfully replicate. These markets demonstrated an impressive 70% to 73% accuracy rate in predicting binary replication outcomes, performing slightly better than survey beliefs and significantly better than random chance. The critical precondition, however, was the existence of a highly orchestrated, highly funded administrative body that executed the physical replication experiments within a defined timeframe to settle the bets.

In cybersecurity, Bug Bounty programs operate as a prominent form of staked verification. Hackers submit vulnerabilities, and companies pay out rewards based on the severity of the exploit. The mechanism delivers high-quality audits, but it requires rapid resolution infrastructure. Programs that succeed maintain aggressive Service Level Agreements (SLAs), aiming for first responses within hours and full resolution and payout within 24 to 48 hours. When resolution is slow or ambiguous, the mechanism degenerates rapidly: top-tier researchers abandon the program due to poor ROI, and the queue fills with low-quality, automated "beg-bounty" spam.

**The Settlement Latency Finding:** Staked verification functions optimally only when settlement is rapid and unambiguous. In replication markets, the settlement was guaranteed by an external grant funding a specific experiment; in bug bounties, it is guaranteed by the immediate execution of a Proof-of-Concept exploit. In the physics-kernel, the settlement of a theoretical claim (e.g., a falsification condition firing or a complex formalization landing) may take years, decades, or may never happen at all. Betting markets and staking ledgers degenerate entirely under indefinite settlement horizons due to the time-value of capital and the evaporation of

liquidity. Without an artificial, short-term proxy for settlement (such as a purely mathematical formalization landing, rather than empirical falsification), staked verification in the kernel will collapse and is strongly advised against.

## 7. The Affordance Map

The following map ranks the candidate affordances derived from the survey. Ranking is calculated heuristically based on the product of home-field evidence of value and transfer plausibility, divided by the implementation cost class.

*Cost Classes:*

- **Read:** A pure query over the existing graph (I9); free to experiment immediately.
- **Periphery:** Requires a new peripheral tool or LLM prompting wrapper.
- **Substrate:** Requires a fundamental change to the core kernel architecture.

| Affordance                                 | Exploited Invariants | Precedent Field & Tool                     | Home-field Evidence                                                       | Transfer Preconditions                     | Earliest Failure Mode in Kernel                                                         | Cost Class | Verdict          |
|--------------------------------------------|----------------------|--------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------|------------|------------------|
| 1. <b>Materiality-Weighted Audit Queue</b> | I1, I5, I9           | Accounting (Audit Sampling / Materiality)  | Reduces unnecessary audit overhead; focuses attention only on risk.       | Quantifiable downstream dependency metric. | Modest corpus scale makes dependency cones too shallow to differentiate priority.       | Read       | <b>CONFIRMED</b> |
| 2. <b>Dynamic Shepardizing</b>             | I2, I3, I5, I9       | Law (Shepard's Citations)                  | Prevents reliance on overruled precedent; foundational to legal research. | Graph-based rests_on tracking.             | LLM fails to recognize the semantic nuance of <i>why</i> the upstream claim was killed. | Read       | <b>CONFIRMED</b> |
| 3. <b>Topological Crucial Isolation</b>    | I1, I3, I5, I6, I9   | Argumentation Theory (Dung) & DB Semirings | Computes logical extensions over complex attack graphs.                   | Binary attack/defense topology.            | Contradictions are too ambiguous to mathematically isolate a single axiomatic root.     | Read       | <b>PLAUSIBLE</b> |
| 4. <b>Graph-Level Linting</b>              | I1, I2, I9           | Software Engineering (Static)              | Reduces Diff Authoring                                                    | Strict, deterministic formatting           | "Warning fatigue" causes the                                                            | Periphery  | <b>CONFIRMED</b> |

| Affordance                              | Exploited Invariants | Precedent Field & Tool                   | Home-field Evidence                                        | Transfer Preconditions                          | Earliest Failure Mode in Kernel                                                    | Cost Class | Verdict                 |
|-----------------------------------------|----------------------|------------------------------------------|------------------------------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------|------------|-------------------------|
|                                         |                      | Analysis)                                | Time by 14-33%.                                            | and logic rules.                                | human director to ignore the linter outputs.                                       |            |                         |
| <b>5. Incremental Synthesis Views</b>   | I5, I7, I9           | DB Theory (Incremental View Maintenance) | Massively reduces computational overhead for data updates. | Transform dependencies mapped to inputs.        | Re-prompting the LLM with delta updates leads to hallucinated or disjointed prose. | Periphery  | <b>MARGINAL</b>         |
| <b>6. Stepping-Stone Recovery</b>       | I3, I4, I9           | Evolutionary Comp. (MAP-Elites)          | Bypasses deceptive landscapes via low-fitness archives.    | Search space defined by behavioral descriptors. | Re-evaluating dead claims consumes too much LLM API budget.                        | Periphery  | <b>NEEDS REFINEMENT</b> |
| <b>7. Adversarial Epistemic Forking</b> | I6, I8               | Open Source (Fork-diff analysis)         | Enables concurrent decentralized exploration.              | Serialization of full state.                    | Single human director lacks the bandwidth to manage multiple divergent worldviews. | Substrate  | <b>MARGINAL</b>         |
| <b>8. Staked Verification</b>           | I1, I6, I8           | Replication Markets / Bug Bounties       | 73% accuracy in predicting replication success.            | Rapid, unambiguous settlement events.           | Settlement latency stretches to years, causing market to freeze or degenerate.     | Substrate  | <b>DEAD</b>             |

## The Three Free Experiments

Because they rely entirely on I9 reads over the existing graph without requiring new infrastructure, the program can test the top three affordances immediately:

1. **Dynamic Shepardizing:** Write a query that traces the contamination path of a newly generated LLM claim upward. If any node in its rests\_on lineage carries a "killed" status, mechanically reject the claim before it reaches the director's screen.
2. **Materiality-Weighted Audit Queue:** Write a query that scores all open, un-audited claims by calculating the total count of downstream nodes dependent upon them. Present the director with a review dashboard sorted strictly by this materiality score.
3. **Topological Crux Isolation:** For any two claims locked in a contradicts relation (I6), write a query that traverses their respective audit trails down to the base axioms. Output the diff of their axiomatic roots to the human director as the precise mathematical crux of the disagreement.

## Judgment Calls

The ranking heavily penalizes *Staked Verification* (Verdict: DEAD) despite its strong home-field evidence in the social sciences. The judgment call here relies on the rigorous precondition analysis: replication markets function exclusively because projects like SCORE guaranteed a physical replication trial to settle the bet. Without a guaranteed, short-term mechanism to falsify a physics claim, the staking mechanism lacks the necessary velocity to align incentives. Conversely, the *Materiality-Weighted Audit Queue* is ranked first. While it borrows from accounting, the translation to the verifier economy of an LLM system is nearly structurally identical. The human director's attention is the system's most expensive and scarce resource; computing dependency cones to protect that resource is structurally identical to calculating financial risk thresholds, making the isomorphism exceptionally robust.

## 8. Literature Bibliography

- [1] Green, T. J., Karvounarakis, G., & Tannen, V. (2007). Provenance Semirings. PODS '07. [PEER-REVIEWED]
- [2] Green, T. J., et al. (2007). Provenance Semirings (Abstract/Detail). [PEER-REVIEWED]
- [4] Green, T. J., et al. (2007). "We show that relational algebra calculations..." PODS '07. [PEER-REVIEWED]
- [6] Ives, Z., et al. (2007). Abstracting the structure of calculations. PROPR '07. [PEER-REVIEWED]
- [7] Camerer, C. F., et al. (2016). Evaluating the replicability of laboratory experiments in economics. Science. [PEER-REVIEWED]
- [8] Altmejd, A., et al. (2019). Predicting the replicability of social science lab experiments. PLoS ONE. [PEER-REVIEWED]
- [12] Camerer, C. F., et al. (2018). Analysis of survey and prediction market data from large-scale forecasting projects. PLoS ONE. [PEER-REVIEWED]
- [13] Amgoud, L., et al. (2020). Dung abstract argumentation frameworks synthesis tool. AAAI. [PEER-REVIEWED]
- [19] Cochrane Tobacco Addiction Group. (2025). Living systematic review update triggers. [PEER-REVIEWED]
- [25] Baker Tilly. (2024). How materiality is established in an audit. [BLOG-INFORMAL]
- [26] Corporate Finance Institute. Materiality Threshold in Audits. [BLOG-INFORMAL]
- [28] PCAOB. (2024). AS 2105: Consideration of Materiality in Planning and Performing an

Audit. [NUMERICAL]

- [29] Numeric.io. (2024). What is a Materiality Threshold? [BLOG-INFORMAL]
- [31] GitLab. (2024). Vulnerability Triage and Risk Analysis. [BLOG-INFORMAL]
- [32] Nullify.ai. (2024). Capabilities: Triage and Risk Ranking. [BLOG-INFORMAL]
- [37] Wikipedia. Citation Index (Shepard's Citations). [FOLKLORE]
- [38] Wouters, P. (1998). Stumbling over the Citation Concept. [PEER-REVIEWED]
- [41] Benjamin, S., et al. (2024). Shepard's and Judicial Reasoning. Emory Law Journal. [PEER-REVIEWED]
- [42] Dronedesk. (2024). Aviation Safety Reporting System. [BLOG-INFORMAL]
- [43] NASA ASRS. (2024). ASRS Program Overview. [NUMERICAL]
- [44] National Academies of Sciences. Aviation Safety Reporting System. [PEER-REVIEWED]
- [45] Paradis, et al. (2021). NASA ASRS Database Machine Learning Analysis. [PEER-REVIEWED]
- [46] FAA. (2024). Hazardous Materials Incident Reporting. [BLOG-INFORMAL]
- [48] Emergent Mind. (2024). MAP-Elites Algorithm. [BLOG-INFORMAL]
- [49] MDPI. (2024). MAP-Elites variants and taxonomy. [PEER-REVIEWED]
- [52] Anne, M., & Mouret, J.-B. (2023). Multi-Task Multi-Behavior MAP-Elites. [PEER-REVIEWED]
- [54] Sottile, M., et al. (2026). LLM-driven theorem provers. [PREPRINT]
- [55] Intelligence Strategy. (2024). Hilbert's problems as generative infrastructure. [BLOG-INFORMAL]
- [56] ResearchGate. (2024). Strategic Horizons for the Mathematical Sciences. [PREPRINT]
- [59] Baanen, A., et al. (2025). Growing Mathlib: maintenance of a large scale mathematical library. [PREPRINT]
- [60] Baanen, A., et al. (2025). Growing Mathlib. [PEER-REVIEWED]
- [62] Baanen, A., et al. (2025). The deprecation system in Mathlib. [PREPRINT]
- [64] FrenzyMath. (2024). Automated framework for tackling research-level mathematical problems. [BLOG-INFORMAL]
- [66] Emory Law Journal. (2024). Wikipedia dispute resolution and evidence graphs. [PEER-REVIEWED]
- [67] Wikipedia Signpost. (2023). Dispute resolution Wikipedia contested claim machinery. [FOLKLORE]
- [68] Emerald Insight. (2024). The colonization of Wikipedia. [PEER-REVIEWED]
- [73] GitConnected. (2024). Code Review in the Age of AI. [BLOG-INFORMAL]
- [75] Dev.to. (2024). Automated code review benefits. [BLOG-INFORMAL]
- [76] Meta. (2025). Diff Authoring Time (DAT) approach to measuring software development productivity. [PREPRINT]
- [77] AugmentCode. (2024). Autonomous Quality Gates. [BLOG-INFORMAL]
- [78] CodeRabbit. (2024). Code reviews made easy. [BLOG-INFORMAL]
- [80] SwarmSync.ai. (2024). Bug bounty settlement latency. [BLOG-INFORMAL]
- [83] HackerOne. (2017). 5 Hacker-Powered Trends. [NUMERICAL]
- [84] HackerOne. (2025). Hyperledger Bug Bounty. [NUMERICAL]
- [85] The Startup. (2024). Bootstrapping a Bug Bounty Program. [BLOG-INFORMAL]
- [88] Galois. (2024). What are formal methods. [BLOG-INFORMAL]
- [89] Galois. (2024). REVERSER project. [BLOG-INFORMAL]
- [94] GRADEpro. (2024). Handbook: Overview of the GRADE approach. [NUMERICAL]

- [95] GRADEpro. (2024). Overview of the GRADE approach. [NUMERICAL]
- [97] GRADEpro. (2024). GRADEing outcomes step by step. [NUMERICAL]
- [98] GRADEpro. (2024). Inconsistency in GRADE. [NUMERICAL]
- [100] CASRAI. (2024). Living systematic review. [FOLKLORE]
- [101] Cochrane. (2024). Living systematic review triggers. [PEER-REVIEWED]
- [103] Eppi. (2025). Searching and evidence surveillance. [PEER-REVIEWED]
- [104] Cochrane LSR author teams. (2021). Living systematic reviews in the COVID-19 pandemic. [PEER-REVIEWED]
- [105] LSR Authors. (2023). Living systematic review process. [PEER-REVIEWED]
- [106] Machine Learning Made Simple. (2026). The verifier economy. [BLOG-INFORMAL]
- [107] Machine Learning Made Simple. (2026). How to break the memory wall. [BLOG-INFORMAL]
- [128] CodeQL/GitHub. (2024). Tracking Analysis Violations. [PEER-REVIEWED]
- [129] Albany Scholars Archive. (2024). Warning fatigue. [PEER-REVIEWED]
- [130] ResearchGate. (2024). Static Code Analysis in Software Development. [PEER-REVIEWED]
- [133] ICSE. (2013). Why don't software developers use static analysis tools to find bugs? [PEER-REVIEWED]
- [145] Baanen, A., et al. (2025). Growing Mathlib (Deprecated). [PREPRINT]
- [151] Replication Index. (2021). Open Science Collaboration. [BLOG-INFORMAL]
- [153] Replication Index. (2021). OSC Prediction Markets. [BLOG-INFORMAL]
- [155] Altmejd, A., et al. (2019). Predicting the replicability of social science lab experiments. [PREPRINT]
- [156] Wu, et al. (2021). Systematizing Confidence in Open Research and Evidence. [PEER-REVIEWED]
- [157] Databricks. (2026). Incremental view maintenance. [PREPRINT]
- [158] He, Hao, et al. (2024). Incremental View Maintenance: An Algorithmic Approach. [PEER-REVIEWED]
- [159] Vidal, M. P., et al. (2008). A MAPPING-DRIVEN APPROACH FOR SQL/XML VIEW MAINTENANCE. [PEER-REVIEWED]
- [160] Berkeley. (2012). Maintaining Materialized Views. [PEER-REVIEWED]
- [162] UTA. (2024). Incremental maintenance of materialized XML views. [PEER-REVIEWED]
- [170] Alfano, et al. (2025). Dung's abstract argumentation characteristic function. [PEER-REVIEWED]
- [171] Yu, et al. (2026). Agent defense in abstract argumentation semantics. [PEER-REVIEWED]
- [172] Uni.lu. (2024). Dung's abstract argumentation theory. [PEER-REVIEWED]
- [173] IRIT. (2024). Dung's abstract argumentation system. [PREPRINT]
- [174] ICR. (2005). Dung's abstract argumentation characteristic function. [PEER-REVIEWED]
- [175] Xu, Y., & Cayrol, C. (2024). Initial Sets in Abstract Argumentation Frameworks. [PEER-REVIEWED]
- [176] De & Omori. (2024). On a four-valued logic of formal inconsistency. [PEER-REVIEWED]
- [177] Grokipedia. (2024). Four-valued logic. [FOLKLORE]
- [178] IEP. (2024). Paraconsistent Logic. [FOLKLORE]
- [179] Wikipedia. (2024). Paraconsistent logic. [FOLKLORE]

- [181] GRADE guidelines. (2024). Rating the quality of evidence - Inconsistency. [PEER-REVIEWED]
- [187] Algorithm Afternoon. (2024). MAP-Elites. [BLOG-INFORMAL]
- [188] RLVS. (2021). Novelty search concept. [BLOG-INFORMAL]
- [189] Frontiers in Robotics and AI. (2016). Quality diversity algorithms. [PEER-REVIEWED]
- [190] Dominated Novelty Search. (2025). MAP-Elites Grid. [PREPRINT]
- [191] MIT Press. (2024). Preliminary Analysis of Simple Novelty Search. [PEER-REVIEWED]

## Works cited

1. GRADE handbook - GRADEpro, <https://grade.pro.org/handbook/> 2. Overview of the GRADE approach, <https://book.grade.pro.org/guideline/overview-of-the-grade-approach> 3. GRADEing outcomes step-by-step - GRADEpro, <https://help.grade.pro.org/support/solutions/articles/204000074829-gradeing-outcomes-step-by-step> 4. GRADE Book - Inconsistency, <https://book.grade.pro.org/guideline/inconsistency> 5. GRADE guidelines: 7. Rating the quality of evidence - Inconsistency - ResearchGate, [https://www.researchgate.net/publication/51534829\\_GRADE\\_guidelines\\_7\\_Rating\\_the\\_quality\\_of\\_evidence\\_-\\_Inconsistency](https://www.researchgate.net/publication/51534829_GRADE_guidelines_7_Rating_the_quality_of_evidence_-_Inconsistency) 6. Growing Mathlib: maintenance of a large scale mathematical library - arXiv, <https://arxiv.org/html/2508.21593v2> 7. Automated Conjecture Resolution with Formal Verification - FrenzyMath, <https://frenzymath.com/blog/conjecture/> 8. Triage | GitLab Docs, [https://docs.gitlab.com/user/application\\_security/triage/](https://docs.gitlab.com/user/application_security/triage/) 9. Triage | Nullify, <https://docs.nullify.ai/capabilities/triage> 10. How materiality is established in an audit or a review - Baker Tilly, <https://www.bakertilly.com/insights/how-materiality-is-established-in-an-audit-or-a-review> 11. Minding the Gaps: How to Calculate Materiality Thresholds in Accounting | Numeric, <https://www.numeric.io/blog/materiality-threshold> 12. Your Guide to Aviation Safety Reporting Systems - The Dronedesk Blog, <https://blog.dronedesk.io/aviation-safety-reporting/> 13. Read "Accident Precursor Analysis and Management: Reducing Technological Risk Through Diligence" at NAP.edu, <https://www.nationalacademies.org/read/11061/chapter/5> 14. NASA ASRS Program Briefing, [https://asrs.arc.nasa.gov/docs/ASRS\\_ProgramBriefing.pdf](https://asrs.arc.nasa.gov/docs/ASRS_ProgramBriefing.pdf) 15. Mandatory and Voluntary Incident Reporting - Federal Aviation Administration, <https://www.faa.gov/hazmat/incident-reporting> 16. Analyzing Unmanned Aircraft System (UAS) Incidents from NASA ASRS Data Using Unsupervised Machine Learning - ScholarWorks@UARK, <https://scholarworks.uark.edu/cgi/viewcontent.cgi?article=1022&context=elcsuht> 17. MAP-Elites: Quality-Diversity Search - Emergent Mind, <https://www.emergentmind.com/topics/map-elites-algorithm> 18. Generational Adversarial MAP-Elites for Multi-Agent Game Illumination - MIT Press Direct, <https://direct.mit.edu/isal/proceedings-pdf/isal2025/37/31/2567197/isal.a.859.pdf> 19. Quality Diversity: A New Frontier for Evolutionary Computation, <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2016.00040/full> 20. Dominated Novelty Search: Rethinking Local Competition in Quality-Diversity - arXiv, <https://arxiv.org/html/2502.00593v1> 21. Map Elites - Algorithm Afternoon, [https://algorithmafternoon.com/novelty/map\\_elites/](https://algorithmafternoon.com/novelty/map_elites/) 22. Preliminary Analysis of Simple Novelty Search | Evolutionary Computation - MIT Press Direct, <https://direct.mit.edu/evco/article/32/3/249/116787/Preliminary-Analysis-of-Simple-Novelty-Search>

h 23. Large Language Model-Driven Formal Mathematics at the Research Frontier - arXiv, <https://arxiv.org/pdf/2607.07779> 24. What is Good Mathematics? - European Nexus for Strategic Intelligence, <https://www.intelligencestrategy.org/blog-posts/what-is-good-mathematics> 25. Strategic Horizons for the Mathematical Sciences - ResearchGate, [https://www.researchgate.net/publication/397706774\\_Strategic\\_Horizons\\_for\\_the\\_Mathematical\\_Sciences](https://www.researchgate.net/publication/397706774_Strategic_Horizons_for_the_Mathematical_Sciences) 26. Citation index - Wikipedia, [https://en.wikipedia.org/wiki/Citation\\_index](https://en.wikipedia.org/wiki/Citation_index) 27. The Creation of the Science Citation Index, [https://webdoc.sub.gwdg.de/ebook/s/2001/chf/www.chemheritage.org/historicals-services/asis\\_documents/asis98\\_wouters.pdf](https://webdoc.sub.gwdg.de/ebook/s/2001/chf/www.chemheritage.org/historicals-services/asis_documents/asis98_wouters.pdf) 28. Partisan Panel Composition and Reliance on Earlier Opinions in the Circuit Courts - Emory Law Scholarly Commons, <https://scholarlycommons.law.emory.edu/cgi/viewcontent.cgi?article=1245&context=faculty-articles> 29. Provenance Semirings | Paper - Ryan Marcus, [https://rmarcus.info/dbscholar/papers/1413?in\\_page=2&out\\_page=1](https://rmarcus.info/dbscholar/papers/1413?in_page=2&out_page=1) 30. (PDF) Provenance Semirings - ResearchGate, [https://www.researchgate.net/publication/221559651\\_Provenance\\_Semirings](https://www.researchgate.net/publication/221559651_Provenance_Semirings) 31. Provenance Semirings - Computer Science | UC Davis Engineering, <https://web.cs.ucdavis.edu/~green/papers/pods07.pdf> 32. Provenance Semirings - University of Pennsylvania, <https://www.cis.upenn.edu/~plclub/propr/greg-slides.pdf> 33. Extending Abstract Argumentation Frameworks with Knowledge Bases - KR Proceedings, <https://proceedings.kr.org/2025/3/kr2025-0003-alfano-et-al.pdf> 34. Agent defense in abstract argumentation: Semantics and principle-based analysis - Liuwen YU, <https://yuliuwen.github.io/assets/files/yu-et-al-2026-agent-defense-in-abstract-argumentation-semantics-and-principle-based-analysis.pdf> 35. Representation Equivalences Among Argumentation Frameworks - ORBilu, <https://orbilu.uni.lu/bitstream/10993/36359/1/main.pdf> 36. Characterizing change in abstract argumentation systems - IRIT, <ftp://ftp.irit.fr/IRIT/ADRIA/PapersDupin/DraftBisquertandco.pdf> 37. A Logic of Abstract Argumentation - ICR, <https://icr.uni.lu/leonvandertorre/papers/argmas05.pdf> 38. Initial Sets in Abstract Argumentation Frameworks - CEUR-WS.org, <https://ceur-ws.org/Vol-1811/paper6.pdf> 39. (PDF) On a four-valued logic of formal inconsistency and formal undeterminedness - ResearchGate, [https://www.researchgate.net/publication/366027187\\_On\\_a\\_four-valued\\_logic\\_of\\_formal\\_inconsistency\\_and\\_formal\\_undeterminedness](https://www.researchgate.net/publication/366027187_On_a_four-valued_logic_of_formal_inconsistency_and_formal_undeterminedness) 40. Four-valued logic - Grokipedia, [https://grokipedia.com/page/Four-valued\\_logic](https://grokipedia.com/page/Four-valued_logic) 41. Paraconsistent Logic | Internet Encyclopedia of Philosophy, <https://iep.utm.edu/para-log/> 42. Paraconsistent logic - Wikipedia, [https://en.wikipedia.org/wiki/Paraconsistent\\_logic](https://en.wikipedia.org/wiki/Paraconsistent_logic) 43. Enzyme: Incremental View Maintenance for Data Engineering - arXiv, <https://arxiv.org/html/2603.27775v1> 44. Materialized Views Contents - Berkeley, <https://dsf.berkeley.edu/cs286/papers/mv-fntdb2012.pdf> 45. A MAPPING-DRIVEN APPROACH FOR SQL/XML VIEW MAINTENANCE - SciTePress, <https://www.scitepress.org/Papers/2008/17119/17119.pdf> 46. Incremental Maintenance of Materialized XML Views - Leonidas Fegaras, <https://lambda.uta.edu/views10.pdf> 47. How to improve code quality without burning out - CodeRabbit, <https://www.coderabbit.ai/blog/code-reviews-made-easy-how-to-improve-code-quality> 48. (PDF) Different Approach Analysis for Static Code in Software Development - ResearchGate, [https://www.researchgate.net/publication/348805881\\_Different\\_Approach\\_Analysis\\_for\\_Static\\_Code\\_in\\_Software\\_Development](https://www.researchgate.net/publication/348805881_Different_Approach_Analysis_for_Static_Code_in_Software_Development) 49. Code Review in the Age of AI: Why Review Matters More Now, Not Less | by The\_Architect, <https://levelup.gitconnected.com/code-review-in-the-age-of-ai-why-review-matters-more-now-not-less-5c3ac61656c8> 50. What's DAT? Three Case Studies of Measuring Software Development



Productivity at Meta With Diff Authoring Time - arXiv, <https://arxiv.org/html/2503.10977v2> 51. Opting Out: Over-alerting and Warning Fatigue in the era of Wireless Emergency Alerts - Scholars Archive, [https://scholarsarchive.library.albany.edu/cgi/viewcontent.cgi?article=1019&context=ehc\\_fac\\_scholar](https://scholarsarchive.library.albany.edu/cgi/viewcontent.cgi?article=1019&context=ehc_fac_scholar) 52. Why don't software developers use static analysis tools to find bugs? - ResearchGate, [https://www.researchgate.net/publication/261192385\\_Why\\_don't\\_software\\_developers\\_use\\_static\\_analysis\\_tools\\_to\\_find\\_bugs](https://www.researchgate.net/publication/261192385_Why_don't_software_developers_use_static_analysis_tools_to_find_bugs) 53. Wikitruth Through Wikiorder - Emory Law Scholarly Commons, <https://scholarlycommons.law.emory.edu/cgi/viewcontent.cgi?article=1360&context=elj> 54. Wikipedia:Wikipedia Signpost/2023-11-20/Recent research, [https://en.wikipedia.org/wiki/Wikipedia:Wikipedia\\_Signpost/2023-11-20/Recent\\_research](https://en.wikipedia.org/wiki/Wikipedia:Wikipedia_Signpost/2023-11-20/Recent_research) 55. The colonization of Wikipedia: evidence from characteristic editing behaviors of warring camps | Journal of Documentation - Emerald Insight, <https://www.emerald.com/jd/article/79/3/784/194919/The-colonization-of-Wikipedia-evidence-from> 56. Growing Mathlib: maintenance of a large scale mathematical library - ResearchGate, [https://www.researchgate.net/publication/395125519\\_Growing\\_Mathlib\\_maintenance\\_of\\_a\\_large\\_scale\\_mathematical\\_library](https://www.researchgate.net/publication/395125519_Growing_Mathlib_maintenance_of_a_large_scale_mathematical_library) 57. Findings from a Cochrane Living Systematic Review - Tobacco Online Policy Seminar (TOPS), <https://www.tobaccopolicy.org/pdfs/IQV%20TOPS%20July%202025.pdf> 58. Living systematic review - CASRAI, <https://casrai.org/dictionary/term/living-systematic-review> 59. Chapter 22: Prospective approaches to accumulating evidence - Cochrane, <https://www.cochrane.org/authors/handbooks-and-manuals/handbook/current/chapter-22> 60. Living systematic reviews: what, why, when and how?, [https://eppi.ioe.ac.uk/CMS/Portals/0/NIHR-ESGs-BPWG-LES\\_Webinar-Hodder-04-09-2025\\_Slides.pdf](https://eppi.ioe.ac.uk/CMS/Portals/0/NIHR-ESGs-BPWG-LES_Webinar-Hodder-04-09-2025_Slides.pdf) 61. Methodological challenges for living systematic reviews conducted during the COVID-19 pandemic: A concept paper - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8435072/> 62. Optimizing process and methods for a living systematic review: 30 search updates and three review updates later - PubMed, <https://pubmed.ncbi.nlm.nih.gov/38043829/> 63. The Most Expensive Lie in AI. Billions are being spent to teach... | by Devansh - Medium, <https://machine-learning-made-simple.medium.com/the-most-expensive-lie-in-ai-26dc330f92cf> 64. How to break the memory wall for large-scale Graph Neural Networks - Devansh - Medium, <https://machine-learning-made-simple.medium.com/how-to-break-the-memory-wall-for-large-scale-graph-neural-networks-1bb11972cdd0> 65. Materiality Threshold in Audits - Overview and Methods - Corporate Finance Institute, <https://corporatefinanceinstitute.com/resources/accounting/materiality-threshold-in-audits/> 66. Automated Code Review: Benefits, Tools & Implementation (2026 Guide) - DEV Community, <https://dev.to/cpave3/automated-code-review-benefits-tools-implementation-2026-guide-5dgd> 67. Autonomous Quality Gates: AI-Powered Code Review, <https://www.augmentcode.com/learn/autonomous-quality-gates-ai-powered-code-review> 68. Predicting the replicability of social science lab experiments | PLOS One - Research journals, <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0225826> 69. Predicting replicability—Analysis of survey and prediction market data from large-scale forecasting projects - Stockholm School of Economics, [https://research.hhs.se/view/pdfCoverPage?instCode=46SSOE\\_INST&filePid=1361922380006056&download=true](https://research.hhs.se/view/pdfCoverPage?instCode=46SSOE_INST&filePid=1361922380006056&download=true) 70. Tag Archives: Open Science Collaboration - Replicability-Index, <https://replicationindex.com/tag/open-science-collaboration/> 71. Prediction Markets of Replicability, <https://replicationindex.com/2021/05/10/osc-pm/> 72. Systematizing Confidence in

Open Research and Evidence (SCORE) - ODU Digital Commons,  
[https://digitalcommons.odu.edu/context/computerscience\\_fac\\_pubs/article/1286/viewcontent/Wu\\_2021\\_SystematizingConfidenceinOpenResearchandEvidenceOCR.pdf](https://digitalcommons.odu.edu/context/computerscience_fac_pubs/article/1286/viewcontent/Wu_2021_SystematizingConfidenceinOpenResearchandEvidenceOCR.pdf) 73. 5 Hacker-Powered Trends You Need to Know About - HackerOne,  
<https://www.hackerone.com/blog/5-hacker-powered-trends-you-need-know-about> 74. Bootstrapping a Bug Bounty Program | by Cheston Lee | The Startup - Medium,  
<https://medium.com/swlh/bootstrapping-a-bug-bounty-program-cb0d00518045> 75. Security Controls for Agent Orchestration - SwarmSync, <https://swarmsync.ai/security> 76. Linux Foundation Decentralized Trust - Bug Bounty Program - HackerOne,  
<https://hackerone.com/hyperledger>